

第九讲 整数安全

一、概述

1、整数

整数已日益成为 C 和 C++ 程序中漏洞的源泉

在大多数 C 和 C++ 软件系统的开发中都没有系统地进行整数范围检查

因整数导致的安全缺陷问题肯定存在

→ 其中一些缺陷很可能会成为漏洞。

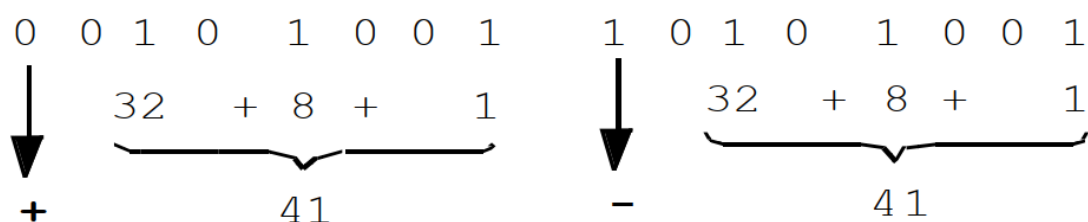
当程序对一个整数求出了一个非期望中的值，软件漏洞就出现了。

2、整数表示方法

(1) 原码表示法

利用最高位表示数值的符号：0 正 1 负

余下的所有低位表示值的大小

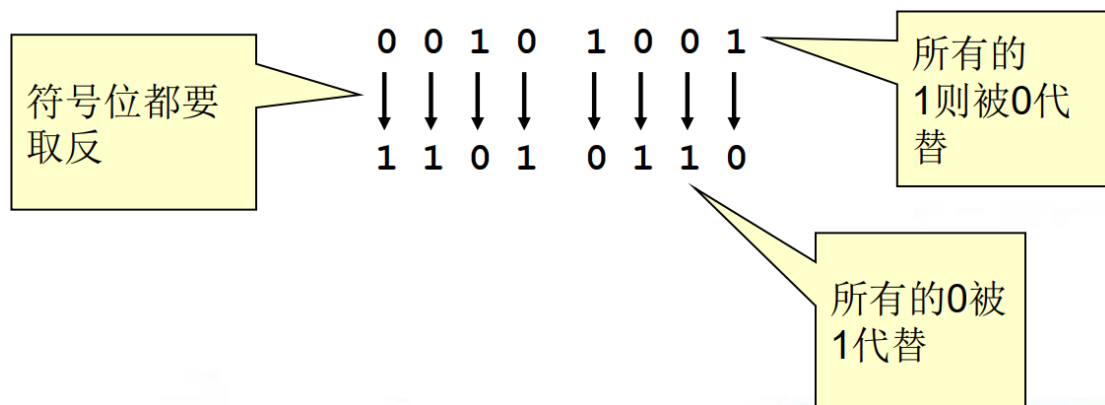


如果最高位未置位，表示+41，如果最高位被置位，则表示-41。

(2) 反码表示法

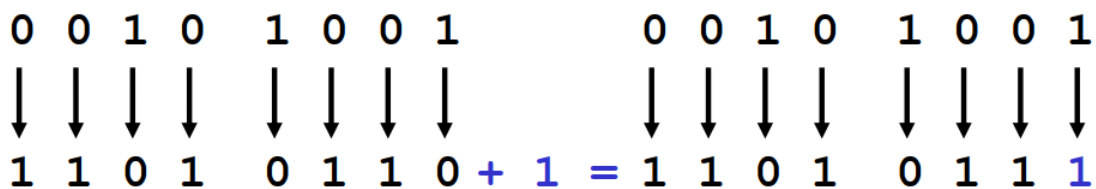
由于实现原码表示法所需的电路过于复杂，因此人们后来采用反码表示法取而代之。

将一个整数值的一位取反，就得到于其对应的负数。



(3) 补码表示法

补码表示法的负数是在反码表示法的结果末位加1而得。



补码表示法对0只有+0一种表示。

最高位仍然是符号位。

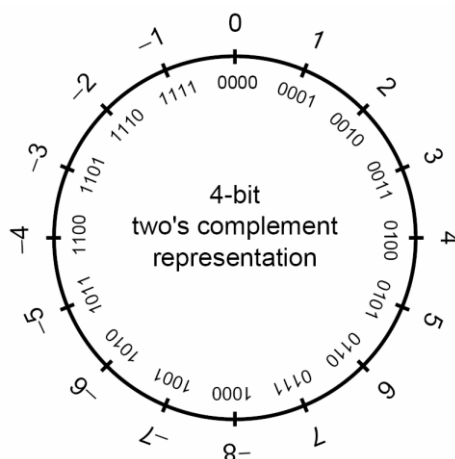
正数的补码表示法则与原码表示法相同。

(4) 带符号整数

带符号整型用来表示正值和负值

在一个使用补码表示法的计算机上，带符号整数的取值范围是

-2^{n-1} 到 $2^{n-1} - 1$ 。

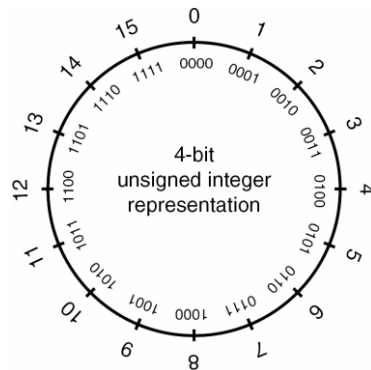


(5) 无符号整数

无符号整数的取值范围：最小值为 0，最大值依赖于该类型所占位数的大小

如果该类型所占位数的大小为 n ，那么最大值即为 $2^n - 1$

任一种带符号整型都有对应的无符号整型。



(6) 整数类型

① 标准类型

标准整型包含的类型如下（以长度非递减的顺序排列）：

→ signed char → short int → int → long int → long long int

② 扩展整型

扩展整型由具体实现定义，包含如下类型：

→ int#_t, uint#_t 其中 # 表示该类型的精确宽度

→ int_least#_t, uint_least#_t 其中 # 表示该类型的最小宽度

→ int_fast#_t, uint_fast#_t 其中 # 表示在保证指定的最小宽度的前

提下具有最快处理速度的宽度值

→ intptr_t, uintptr_t 表示其宽度足够保存对象指针类型的整型。

→ intmax_t, uintmax_t 表示最宽的整型。

③ 平台相关的整型

厂商通常会提供一些平台相关的整数类型

例如微软 Windows API 就定义了大量的整型：

→ `__int8`, `__int16`, `__int32`, `__int64`

→ A 到 M

→ `BOOLEAN`, `BOOL`

→ `BYTE`

→ `CHAR`

→ `DWORD`, `DWORDLONG`, `DWORD32`, `DWORD64`

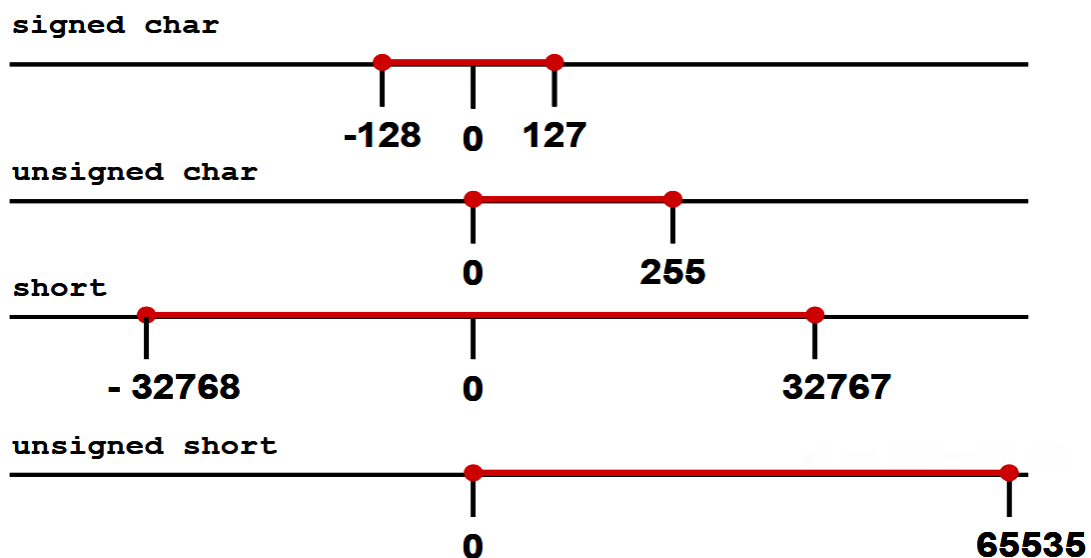
→ `WORD`

→ `INT`, `INT32`, `INT64`

→ `LONG`, `LONGLONG`, `LONG32`, `LONG64`

→ Etc.

(7) 整数取值范围



一个整数类型的最大值和最小值取决于

→ 该类型的表示法

→是否带符号

→分配的内存位数大小

二、整形转换

1、整型转换

●在 C 和 C++ 中，类型转换既可能作为转型（cast）操作的结果显式发生，也可能因为某个操作的需要而隐式发生。

●整型转换可能会导致数据丢失或错误的表示。

●隐式转换是 C 语言可以对混合数据类型执行操作能力的结果。

●C99 标准规定了 C 编译器应该如何处理转换操作

→整型提升

→整型转换级别

→普通算术转换

2、整型提升

●在比 int 小的整型进行操作时，它们会被提升。

●如果原始类型的所有值都可以用 int 表示，

→较小的类型会被转换成一个 int

→否则被转换成一个 unsigned int

●整型提升被作为普通算术转换的一个组成部分

→某些自变量表达式

→一元的+、-、和 ~操作符

→移位操作

● 由于整型提升，因此这里 `c1` 和 `c2` 都要被提升到 `int` 类型的大小

```
char c1, c2;
```

```
c1 = c1 + c2;
```

● 然后两个 `int` 类型的数据相加，求和结果被截断以适应 `char` 类型的大小。

● 整型提升主要是为了防止运算过程中中间结果发生溢出而导致算术错误

3、隐式转换

● 1. `char cresult, c1, c2, c3;`

● 2. `c1 = 100;`

● 3. `c2 = 90;`

● 4. `c3 = -120;`

● 5. `cresult = c1 +`

`c1`、`c2`的和超过了signed char类型的最大值

然而，由于发生了整型提升，`c1`、`c2`和`c3`都被转换为整型，因此整个表达式的结果能够被成功地计算出来。

该结果随后被截断，并存储在 `cresult` 中，没有数据丢失

`c1`与`c2`相加

4、整数转换级别

每一种整数类型都有一个相应的整型转换级别（integer conversion rank），决定了转换操作将会如何执行。

5、整数转换级别的规则

- 没有任何两种不同的带符号整型具有相同的级别，即使它们的（内存）表示法相同。

- 低精度的带符号整型的级别比高精度的带符号整型类型的级别低。

- long long int 类型的级别比 long int 高，long int 的级别比 int 高，int 的级别比 short int 高，short int 的级别比 signed char 高。

- 无符号整型的级别与对应的带符号整型的级别相同。

(1) 从无符号整型转换

- 从较小的无符号整型转换到较大的无符号整型

- 总是安全的

- 通常通过对其值进行零扩展而完成

- 当一个较大的无符号整型被转换到一个较小的无符号整型的时候

- 较大的值将会被截断

- 低位数据被保留

- 当无符号整型转换到其对应的带符号整型的时候

- 位模式（即所有的位数据）将会被保留，因此没有数据会因此丢失

- 最高位数据变成了符号位

- 如果该符号位被置位，该值的符号和大小都会发生改变。

(2) 带符号整型转换

● 当一个非负的带符号整数被转换为一个相同大小或更大的无符号整型的时候，

→ 值不会发生变化

→ 带符号整数需作符号扩展

● 当一个带符号整数被转换为一个较短的带符号整型的时候，则是通过截断高位完成的

● 当带符号整数转换到无符号整数

→ 其位模式被保留，故不会有数据的丢失

→ 高位失去了符号位的功能

● 如果带符号整型的值非负的话，那它的值不会发生改变。

● 如果其值是负数的话，得到的无符号结果将被求值为一个非常大的带符号整数。

```
1. unsigned int l = ULONG_MAX;
2. char c = -1;
3. if (c == l) {
4.     printf("-1 = 4,294,967,295?\n");
5. }
```

c与1 进行相等性比较

由于整型提升规则发挥作用，导致c被转换为一个无符号的整数，其值为0xFFFFFFFF 或 4,294,967,295

6、带符号或无符号字符

● char 类型既可以是带符号的，也可以是无符号

● 当一个符号位被置位的 signed char 类型的数据被当作整型保存时，其结果就是一个负数。

● 当处理值可能会大于 127 (0x7F) 的字符数据时，对于涉及的字符缓冲区、指针及转型，最好用 unsigned char 代替 char 或 signed char。

三、整数错误及漏洞

1、整数错误情形

(1) 整数溢出

当一个整数被增加超过其最大值或被减小小于其最小值时即会发生整数溢出

带符号和无符号的数都有可能发生溢出：带符号溢出发生于对符号位执行进位时，无符号溢出则发生于当底层表示不再能够表示一个值时。

```
int i;
unsigned int j;

i = INT_MAX; // 2,147,483,647
i++;
printf("i = %d\n", i);

j = UINT_MAX; // 4,294,967,295;
j++;
printf("j = %u\n", j);
```

i=-2,147,483,648

j = 0

```
i = INT_MIN; // -2,147,483,648;
```

```
i--;
```

i=2,147,483,647

```
printf("i = %d\n", i);
```

```
j = 0;
```

```
j--;
```

j = 4,294,967,295

```
printf("j = %u\n", j);
```

(2) 截断错误

截断错误发生于：将一个较大整型的数转换到较小的整型；该数的原值超出较小类型的表示范围。

原值的低位被保留下来而高位则被丢弃。

```
int i = -3;
```

```
unsigned short u;
```

```
u = i;
```

```
printf("u = %hu\n", u);
```

隐式转换为较小的无符号整数

有足够的位来表示值，所以没有发生截断。但是补码表示法被解释为一个大的符号值，所以 **u = 65533**

(3) 符号错误

从无符号整型转换到带符号整型，它们是

→ 相同大小：位模式保留不变；最高位变成符号位

→ 更大：进行符号扩展， 然后才执行转换

→ 更小：保留低位

如果无符号整数的最高位

→ 没被设置：值不变

→ 被设置：变成负值

带符号整型转换到无符号整型，它们是

→ 相同大小：位模式保留不变；最高位变成符号位

→ 更大：进行符号扩展， 然后才执行转换

→ 更小：保留低位

如果带符号整数的值是

→ 非负的：值不变

→ 负的：结果通常是一个很大的正值

●1. `char cresult, c1, c2, c3;`

●2. `c1 = 100;`

●3. `c2 = 90;`

●4. `cresult = c1 + c2;` **c1加 c2 超过了signed char (+127) 的最大值**

当该值赋给一个太小的数据类型的时候，而无法表示结果值的时候。会发生截断错误。

在操作前，把比**int** 小的整型提升为 **int** 或**unsigned int**

2、漏洞

(1) JPEG 例子

基于在处理 JPEG 文件注释域 (comment field) 时存在的实际漏洞

→ JPEG 文件的注释域中包含一个长为两个字节的长度域, 后者用来指示注释域的长度 (也包括该两个字节的长度域本身在内)

→ 为了确定注释字符串的单独长度 (以便进行内存分配), 函数会读取长度域的值并将其减 2。

→ 后函数根据注释的长度加上用于表示终结 null 字符的 1 个字节所得的总长度来分配内存空间。

(2) Integer 整数溢出例子

```
void getComment(unsigned int len, char *src) {  
    unsigned int size;  
  
    size = len - 2;  
  
    char *comment = (char *)malloc(size + 1);  
    memcpy(comment, src, size);  
    return;  
}
```

分配到0字节内存可以成功执行

一个极大的正整数0xffffffff

```
int _tmain(int argc, _TCHAR* argv[]) {  
    getComment(1, "Comment ");  
    return 0;  
}
```

当图像注释的长度域的数值为1时可能会产生溢出

整数溢出条件:

如果计算所得结果无法用带符号整数表示, 那么, 尽管分配程序看上去能够成功地执行, 但实际上它只会分配非常小的内存空间。

应用程序对分配的缓冲区的写操作可能会越界，从而导致基于堆的缓冲区溢出。

(3) 内存分配例子

整数溢出例子也会发生于内存分配 `calloc()` 时，当计算一块内存区域的大小并调用 `calloc()` 或其他内存分配函数来分配内存时可能会引起整数溢出

可能会返回一个小于需求大小的缓存，从而可能会导致后面的缓冲区溢出

以下代码片断可能会产生漏洞：

C: `p = calloc(sizeof(element_t), count);`

C++: `p = new ElementType[count];`

库函数 `calloc()` 接受两个参数：→ 存储元素类型所需要的空间

→ 元素的个数

对于 C++ 的 `new` 操作符的情形，则不需要显式指定元素类型大小

为了计算所需内存的大小，使用元素个数乘以该元素类型所需的单位空间来计算

(4) 符号错误例子

```
#define BUFF_SIZE 10
```

```
int main(int argc, char* argv[]){
```

```
    int len;
```

```
    char buf[BUFF_SIZE];
```

```
    len = atoi(argv[1]);
```

```
    if (len < BUFF_SIZE){
```

```
        memcpy(buf, argv[2], len);
```

```
    }
```

```
}
```

程序接受两个参数：将要被复制的字符串和它的长度

len被声明为带符号整型

argv[1] 也可以是一个负值

一个负值能够绕过该检测

值被认为是无符号的**size_t**类型

负的长度值被解释为一个很大的正整数，从而导致缓冲区溢出

这个漏洞可以通过限制整数 len 为一个有效值来避免

→ 加设一个更有效的范围校验以保证 len 的值在 0 到

BUFF_SIZE 之间

→ 声明为无符号整型 declare as an unsigned integer

➤ 消除在调用函数 memcpy () 时从带符号整型到无符号整型的转换

➤ 防止发生符号错误

(5) 截断：漏洞执行

```
bool func(char *name, long cbBuf) {  
    unsigned short bufSize = cbBuf;  
    char *buf = (char *)malloc(bufSize);  
    if (buf) {  
        memcpy(buf, name, cbBuf);  
        if (buf) free(buf);  
        return true;  
    }  
    return false;  
}
```

cbBuf 用户初始化用于分配 **buf** 内存的 **bufSize**

cbBuf 被声明为 **long** 用于设定 **memcpy()** 操作中的大小参

北京邮电大学

- **cbBuf** 被临时保存在了一个 `unsigned short bufSize` 中
- 不论是 GCC 还是 Visual C++，在基于 IA-32 的编译器上，`unsigned short` 的最大值都是 65 535。而同一平台上 `signed long` 的最大值是 2 147 483 647。
- 任何值位于 65 535 和 2 147 483 647 之间的 **cbBuf** 在进行赋值时都会发生截断错误
- 当 **bufSize** 同时用于调用 `malloc()` 和 `memcpy()` 的时候，只会发生错误并不会产生漏洞
- 由于 **bufSize** 被用于分配缓冲区的大小，而 **cbBuf** 则是用来在调用函数 `memcpy()` 时指定大小，因此，任何在 1 到 2,147,418,112 (2,147,483,647 - 65,535) 字节之间的 **buf** 值都会引起溢出

(6) 非异常的整数逻辑错误

```
int *table = NULL;
```

```
int insert_in_table(int pos, int value){
```

```
    if (!table) {
```

```
        table = (int *)malloc(sizeof(int) * 100);
```

```
    }
```

```
    if (pos > 99) {
```

```
        return -1;
```

```
    }
```

```
    table[pos] = value;
```

```
    return 0;
```

```
}
```

从堆中分配存储空间给数组

pos 小于 99

value 被插入数组的指定位置

对插入位置 pos 缺乏必要的范围检查，因此将会导致一个漏洞

→ 因为 pos 开始时被声明为带符号整数，即传递到函数中的值可正可负

→ 可以捕获下标越界的正值，但是负值却不会被捕获

(7) 其他 C99 整数类型

● 下面的类型有特定的用处：

→ ptrdiff_t 为表示两指针相减的结果的带符号整型

→ size_t 是表示 sizeof 操作符结果的无符号整型。

→ wchar_t 的取值范围可以表示所支持的现场 (locales) 中最大扩展字符集中的所有字符代码。



接受两个字符串类型的参数并且计算它们的总长度（加上结尾空字符占用的1个额外的字节）

```
int main(int argc, char *const *argv) {  
    unsigned short int total;  
    total = strlen(argv[1]) +  
            strlen(argv[2]) + 1;  
    char *buff = (char *) malloc(total);  
    strcpy(buff, argv[1]);  
    strcat(buff, argv[2]);  
}
```

分配足够的内存来存储两个字符

首先将第一个参数复制到缓冲区中，然后将第二个参数连接在其尾部

攻击者可能会提供两个总长度无法用 unsigned short 整数 total 表示的字符串做参数

strlen()函数返回一个 size_t 类型， 一个 IA-32 上的 unsigned long int

- 有因此，lengths+1 的和是一个 unsigned long int.
- 分配给 unsigned short int total 时，值必须被截断
- 且长度的总和大于结果类型的表示范围，它将会被截模截斯

(8) NetBSD 例子

● NetBSD 1.4.2 及之前的版本中都使用了以下形式的整数范围检查：

```
if (off > len - sizeof(typename))  
    goto error;
```

● 这里的 off 和 len 都是带符号整型

- sizeof 操作符返回的是一个无符号整型(size_t)

- 因此整数提升规则要求 len - sizeof(类型名)

- 应该按照无符号整型计算

- 当 len 小于 sizeof 的返回值时

- 减法操作造成下溢并产生一个很大的正值

- 整数范围检查逻辑被绕过

- 一种能够消除此类问题的替代形式的整数范围检查如下所

示：

```
if ((off + sizeof(type-name)) > len)
```

```
    goto error;
```

- 程序员仍然必须保证 off 的值在一个定义的范围之内，以确保加法操作不会导致溢出

四、缓解策略

1、范围检查

- 如果适当地运用类型范围检查，就够消除所有的整型漏洞。

诸如 Pascal 或者 Ada 这些语言。允许对任何标量类型应用范围限制，以形成子类型。以 A 也为例，允许使用 range 关键字来声明对

派生类型的范围约束

```
type day is new INTEGER range 1..31;
```

- 范围约束会被语言运行时强制执行

- C 和 C++ 在强制类型安全性方面并不擅长

```

#define BUFF_SIZE 10
int main(int argc, char* argv[]) {
    unsigned int len;
    char buf[BUFF_SIZE];
    len = atoi(argv[1]);
    if ((0 < len) && (len < BUFF_SIZE) ) {
        memcpy(buf, argv[2], len);
    }
    else
        printf("too much data\n");
}

```

隐式类型检查是将len声明为无符号整数而实现的

显式范围检查数组的上下界

范围检查的解释：

仅仅将 len 声明为无符号整数并不足以完成范围约束，因为它只能约束从 0 到 MAX_INT 的范围。

检查上下界，确保保证不会将越界的值传递给 memcpy () 同时使用隐式和显式检查看上去有些累赘，但是我们推荐这种“健康的偏执式的编程实践”

● 所有外部输入的数据都要进行上下界检查

→ 应该通过接口来强制执行对它们的限制

→ 任何能够限制过大或过小输入的措施，都有助于防止溢出和其他类型范围错误

● 够限制过大或过小输入

● 排版约定

→ 区分代码中的常量和变量

→ 区分受外部影响的变量和拥有良好定义范围的局部变量

2、强类型

- 提供更好的类型检查的方式之一是提供更好的类型定义

- 将一个变量声明为无符号的类型就能够保证该变量不会包含负值

- 这种解决方案不能阻止溢出

- 使用强类型机制 (strong typing)，有助于编译器更有效地识别与范围相关的问题

- 要想定义一个整数存储水处于液态的华氏温度，可以如下方式声明一个变量：

```
unsigned char waterTemperature;
```

- 使用无符号 8 位数来描述 waterTemperature 是足够的，因此它的值的范围是 1 ~ 255

- 水温范围是从华氏 32 度（冰点）到华氏 212 度（沸点）。

然而，使用这个类型既无法阻止溢出，也允许了无效数值（即 1 ~ 31 和 213-255）的使用

3、抽象数据类型

- 解决方案之一是创建一个包含私有数据成员 waterTemperature 的抽象类型，用户不能直接访问该数据成员

- 这些方法中必须提供类型安全机制，以保证 waterTemperature 的值在有效范围之内

- 如果正确地做到了这一点，就不可能再发生整型范围错误了

4、Visual C++ 编译器检查

当一个整数值被赋给较小的整型时，VisualC++ NET 2003 编译器会生成一个警告（C4244）

→ 在警告级别 1，如果类型为_int64 的值被赋值给 unsigned int 类型的话将会生成一个警告

→ 在警告级别 3 和 4，如果一个整形被转换给一个较小的整型将会生成“可能会丢失数据”警告

在警告级别 4 下，以下例子中的赋值就会生成一个 C4244 警告

```
int main() {  
  
    int b = 0, c = 0;  
  
    short a = b + c; // C4244  
  
}
```

5、Visual C++ 运行时检查

● 当一个整数值被赋给较小的整型时，Visual C++ NET2003 编译器会生成一个警告

● /RTC 提供了与 C4244 警告类似的功能，以报告当一个整数被赋值给较小的整型时所导致的数据丢失。

● Visual C 中还包含一个 runtime_checks 的参数，用于禁用或启用 / RTC 设置，但它并不包括用于捕获其他运行时错误（例如溢出）的标志。

6、GCC 运行时检查

● gcc 和 g++ 编译器都包含一个 -ftrapv 的编译选项，该选项对检测运行时整数异常提供了有限的支持

● 按照 gcc 用户手册的说明，这个选项 “为加、减、乘运算的带符号溢出产生陷阱”

● gcc 编译器产生对已有库函数的调用

gcc 运行时系统的一个函数，用于检测带符号 16 位整数加法操作导致的溢出。

```
Wtype __addvs13 (Wtype a, Wtype b) {  
    const Wtype w = a + b;  
    if (b >= 0 ? w < a : w > a)  
        abort ();  
    return w;  
}
```

加法操作被执行，结果与操作数相比较以判断是否发生了溢出。

abort() 被调用，如果

- **b** 非负且 **w < a**
- **b** 为负数并且 **w > a**

北京邮电大学

7、安全的整数操作

● 整数操作可能会导致溢出和数据丢失

● 防止整数漏洞的第一条防线就是进行范围检查

→ 显式

→ 隐式-通过使用强类型达

● 很难保证多个输入变量不被恶意用户操纵，从而导致程序中的某些操作出错

● 另一种可选的或者说是辅助的方式是将每一个操作保护起来

● 这属于一种劳动密集型的方式，实现起来代价很大

● 让输入可能被不确定来源所影响的所有整数操作都使用一个安全整数库

● C 语言兼容库

→ 由 Michael Howard 编写

→ 使用 IA-32 特定机制， 侦测整数溢出条件

8、无符号的加函数

```
in bool UAdd(size_t a, size_t b, size_t *r) {
    asm {
        mov eax, dword ptr [a]
        add eax, dword ptr [b]
        mov ecx, dword ptr [r]
        mov dword ptr [ecx], eax
        jc short j1
        mov al, 1 // 1 is success
        jmp short j2
    j1:
        xor al, al // 0 is failure
    j2:
    };
}

int main(int argc, char *const *argv) {
    unsigned int total;
    if (UAdd(strlen(argv[1]), 1, &total) &&
        UAdd(total, strlen(argv[2]), &total)) {
        char *buff = (char *)malloc(total);
        strcpy(buff, argv[1]);
        strcat(buff, argv[2]);
    } else {
        abort();
    }
}
```

调用函数UAdd（）来计算两个字符串长度的总和时，使用了适当的错误检查机制

9、SafeInt 类

SafeInt 是一个由 David LeBlanc 编写的 C++ 模板类。

在执行操作之前对操作数的值进行测试，以决定是否会导致错误

由于这个类被声明为模板类，因此可以用于任何整数类型

重载了几乎每一个有关的操作符（{}下标索引操作符除外）



SafeInt

变量s1和s2分别被声明为 SafeInt类型

```
int main(int argc, char *const *argv) {
    try{
        SafeInt<unsigned long> s1(strlen(argv[1]));
        SafeInt<unsigned long> s2(strlen(argv[2]));
        char *buff = (char *) malloc(s1 + s2 + 1);
        strcpy(buff, argv[1]);
        strcat(buff, argv[2]);
    }
    catch(SafeIntException err) {
        abort();
    }
}
```

调用+操作符时，使用的是SafeInt类提供的安全版本的+操作符。这个安全版本的操作符保证，倘若结果无效则抛出一个异常

10、整数安全解决方案对比

与 Howard 方法相比， SafeInt 库有好几个优点

→ 比依赖于汇编语言指令实现务全算术操作的 Howard 方法移植性更好

→ 更好的可用性

➢ 算术操作符可以用于常规的内联表达式

➢ SafeInt 使用 C++ 异常处理机制代替 C 风格的返回代码检

查

→ 更好的性能表现(对于启用优化编译选项的应用程序而言)

11、什么时候使用整数安全库

让输入可能被不确定来源所影响的所有整数操作都使用一个安全整数，比如

→ 结构大小

→ 分配的结构个数

安全整数，比如

→ 结构大小

→ 分配的结构个数

函数有两个参数，一个指定了给定结构的大小，一个指定了由非可信来源所能操纵的、应分配的结构个数。这两个值的乘积决定了所分配的内存大小

```
void* CreateStructs(int StructSize, int HowMany) {  
    SafeInt<unsigned long> s(StructSize);  
  
    s *= HowMany;  
  
    return malloc(s.Value());  
}
```

乘法操作很容易引起整型变量的溢出，同时也为缓冲区溢出提供了机会

北京邮电大学

12、什么时候不使用整数安全库

不需要使用安全整数进行操作

→ 被紧密控制的循环

→ 变量不受外部影响

```
void foo() {  
    char a[INT_MAX];  
    int i;  
  
    for (i = 0; i < INT_MAX; i++)  
        a[i] = '\\0';  
}
```

13、GNU 多精度算术库(GMP)

● GMP 是一个用 C 编写的可移植的库，用于对整数、有理数以及浮点数进行任意精度的算术运算

● 需要比 C 基本类型所直接支持的精度更高的精度的应用程序提供尽可能快的算术运算

● GMP 对速度的强调高于简单性和优雅性

● 它使用了复杂的算法，全字（full words）作为基本的算术类型，以及精心优化的汇编代码

14、测试

对输入的整数值进行检查仅仅是一个良好的开端，但这并不能保证对这些整数后继的操作中不会引起溢出或其他错误。

测试也不能提供任何保证

→ 除非程序很简单，要覆盖所有可能的输入是不可能的

→ 如果运用得当的话，测试能够增加代码安全的信心

● 整数漏洞测试应该涵盖所有整型变量的边界条件。

→ 如果类型范围检查被内嵌到代码中，即可测试它们针对整盘上下界是否可以正确地发挥功能。

→ 如果没有包含边界测试，就检测每一个整型所能使用的最大值和最小值。

● 可以用白盒测试决定应该使用什么类型的变量

● 如果得不到源代码，可以用各种类型的最大值和最小值进行测试

15、源代码审计

● 必须对源代码进行审查，以发现可能存在的整数范围错误

● 以下是应该审查的项目：

→ 对整数类型范围进行彻底地检查

→ 根据输入值将被使用的情况检查它们是否被约束在有效范围之内

● 值不能为负的整数（例如作为索引、大小以及循环计数器的整数）应该被声明为无符号型并对上下界进行适当的范围检查

● 对所有来自不确定性来源的整数操作，都使用安全整数库完成

16、著名的漏洞

(1) XDR 库中的整数溢出

由 Sun 发布的 XDR (external data representation) 库中的 xdr_array

() 函数包含一个整型溢出

该漏洞已经导致多个应用程序出现可被远程利用的缓冲区溢出，从而可以执行任意的代码

很多产商都在自己的实现中包含了那些有漏洞的代码

XDR 库为从一个系统进程发送数据到另一系统进程（通常是通过网络连接发送的）提供了平台无关的方法

这样的函数通常被用在远程过程调用(RPC)的实现中，为需要使用公共接口与许多不同种类的系统进行交互的应用层程序员提供了透明性

由 Sun 提供的 XDR 库中的 `xdr_array()` 函数中包含有一个整型溢出，可能会造成大小错误的动态内存分配，从而可能会导致诸如缓冲区溢出等相应问题

(2) Windows DirectX MIDI 库

● `quartz.dll`，中包含一个整数溢出漏洞，可以让攻击者执行任意的代码，或使任何使用了这个库的应用程序崩溃，从而导致拒绝服务

● Windows 操作系统包含了名为 DirectX 和 DirectShow 的多媒体技术

● DirectX 由一组为 Windows 程序提供多媒体支持的底层应用程序接口(APIs)组成

● 在 DirectX 中，DirectShow 技术执行客户端音频／视频的收集、操纵和呈现

● DirectShow 对 MIDI 文件的支持需要一个名为 quartz.dll 的库实现。

● 由于该库没有充分验证 MIDI 文件的 MThd 区中的音轨 (tracks) 值的有效性，一个精心构造的 MIDI 文件就可以导致整数溢出，进而造成堆内存破坏

● 任何使用了 DirectX 或 DirectShow 来处理 MIDI 文件的应用都可能会受到此漏洞的影响

● 尤其值得注意的是，IE 在处理 HTML 文档内嵌的 MIDI 文件时会载入这些有漏洞的库

● 攻击者只需要说服受害人浏览一个嵌入了有害 MIDI 文件的 HTML 文档，就能对受害人实施攻击

● 大量的应用（例如，Outlook, Outlook Express, Eudora, AOL, Lotus Notes, Adobe Photo Deluxe)都使用了 IE 的 HTML 呈现引擎 (WebBrowser ActiveX control) 来解释 HTML 文档

(3) Bash

● GNU 项目 Bourne Again Shell (bash) 是 Unix Bourne Shell(/bin/sh) 的一个替代品。

● 它和标准 Shell 拥有相同的语法，但提供了如作业控制、命令行编辑和历史记录等额外的功能

● 它最流行的使用是在 Linux 上

● bash 1.14.6 以及更早的版本中存在一个漏洞，该漏洞会导致 bash 被欺骗以执行任意的命令

● bash 源代码的 `parse.y` 模块中的 `yy_string_get()` 函数内有一个变量声明错误.

● 该函数负责将用户输入的命令行解析为单独的记号 (tokens)

● .这个错误源于一个名为 `string` 的变量, 它被声明为 `char *` 类型

● `string` 变量用于遍历包含将被解析的命令行的字符串

● 从该指针取回来的字符被存储到一个 `int` 型的变量中

● 对那些默认 `char` 为 `signed char` 的编译器, 当将其值赋给 `int` 变量时, 会进行符号扩展

● 对十进制代码为 255 的字符 (补码形式的 -1), 符号扩展会导致 `int` 变量被赋值为 -1.

● 解析器的其他部分又使用 -1 作为一个命令的结束标志

● 十进制的 255 (八进制的 377) 就被作为一个通过 `-c` 选项提供给 bash 的非意料的命令分隔符

● 例如

→ `bash -c 'ls\377who'`

→ (其中 `\377` 表示一个具有十进制值 255 的单个字符) 将执行两个命令: `ls` 和 `who`